



INTERNSHIP PROJECT REPORT

Advanced Conversational AI & Chatbot Development

Submitted by

Jinesh Khalas

B.Tech (Completed 2nd Year)

JG University, Ahmedabad, Gujarat

Introduction

During this internship, I designed, built, and deployed six AI-powered chatbots. Each one tackled a real-world problem using Natural Language Processing (NLP) and modern Generative AI tools.

Over the course of this project, I went from building basic conversational bots to advanced systems that use Retrieval-Augmented Generation (RAG), work with images and text together, and even understand human emotions in real time.

I handled every part of the development process myself — from collecting and preparing data, to building the AI pipelines, managing databases, and designing the user interfaces.

Background

Large Language Models (LLMs) have changed the way people interact with technology. But standard LLMs have two big limitations — they sometimes make things up (called hallucinations), and they don't have access to private or real-time data.

This internship focused on solving exactly these problems. I built systems based on RAG, which means the AI looks up real facts from a database before answering, so its responses are grounded in actual information.

I worked from Ahmedabad, Gujarat, and built all solutions to run locally on a device — meaning no data is sent to the cloud, costs are kept low, and user privacy is fully protected.

What I Set Out to Learn

- How RAG works and how to build one from scratch
- How to use vector databases like ChromaDB and FAISS for smart search
- How to build AI that can understand both images and text
- How to detect emotions and make AI respond empathetically
- How to build clean web interfaces using Streamlit
- How to run LLMs on local hardware without cloud APIs
- How to make AI that works in multiple languages including Gujarati, Hindi, and Spanish

Task 1: Dynamic Multi-Format Auto-Sync RAG Chatbot

What I Built

I built a fully offline chatbot that can read and answer questions from your local documents — including PDFs, Word files, Excel sheets, PowerPoint files, and plain text. It updates itself automatically whenever a file is added or changed, so you never have to restart it.

How It Works

- The system watches a local folder for any file changes using MD5 hashing (a way to detect if a file's content has actually changed, not just its name).
- Each file type is handled by the right parser — PDFs go through PyPDFLoader, Word files use Docx2txtLoader, and Excel files are converted into readable text by a Pandas-based parser.
- The text is split into smaller chunks and stored in a local database (ChromaDB) with a unique ID to avoid duplicates.
- When you ask a question, the system finds the 5 most relevant chunks and passes them to Llama 3 (running locally via Ollama), which then gives you an answer with source citations.

Problems I Faced & How I Fixed Them

Problem: The chatbot would forget older documents when new ones were added.

Solution: I switched from using OS timestamps (which are unreliable) to MD5 hashing of the actual file content. The hash values are saved in a JSON file, so the system always knows exactly which files it has already processed — even after a restart.

Problem: When profiles of two different people were loaded, the chatbot would mix up their information.

Solution: I added automatic markers to each chunk of text (like '--- START OF BLOCK FROM FILE: [filename] ---'). This taught the model to always trace answers back to the correct source file.

Problem: For simple questions like 'Hi, how are you?', the bot would search the database and reply with 'I don't know based on the context.'

Solution: I set a distance threshold — if the closest database result is too far from the question, the bot skips the database entirely and answers from its general knowledge instead.

Problem: Complex Excel or PowerPoint files would crash the app.

Solution: I added proper error handling (try-except blocks) around the file parsers, so a bad file gets skipped cleanly without taking down the whole app.

Problem: Running two large models at once (Llama 3 and the embedding model) used up all 8GB of my laptop's VRAM and caused crashes.

Solution: I configured Ollama to run only one model at a time and set up an option to use the lighter Llama 3.2 (3B) model, which uses half the memory and is faster.

Result

The chatbot works as a fully offline, multi-document assistant. It correctly separates information from different files, handles general conversation naturally, and runs smoothly even on modest hardware.

Task 2: Advanced Multimodal Gemini Chatbot

What I Built

A chatbot web app that can handle three things: answer text questions, analyze images you upload, and generate images from a text description. It uses Google Gemini for language and vision, and Flux (an open-source model) for image generation.

How It Works

- When you upload a photo, it sends the image + your question to Gemini 2.5 Flash, which reads the image and gives a detailed answer.
- When you type something like 'draw a sunset over mountains', the app detects the generation intent using keyword matching and sends it to an image generation API.
- For everything else, it works as a normal smart chatbot powered by Gemini 2.5 Flash.

Problems I Faced & How I Fixed Them

Problem: Google's free API kept blocking image generation with quota errors.

Solution: I switched the image generation backend to Pollinations AI (which uses the Flux model) — it's free, public, and doesn't have the same restrictions.

Problem: Long image prompts were too big for the API's URL limit and kept getting rejected.

Solution: I built a prompt optimizer that uses Gemini to automatically shorten any prompt over 150 characters into a tight, descriptive summary before sending it.

Problem: Public image servers often timed out, leaving the screen blank or frozen.

Solution: I added a 15-second timeout and a fallback system — if the image server doesn't respond, the app shows a relevant local image from a small offline collection instead.

Problem: The chat UI was crashing with 'KeyError: type' errors and duplicating messages.

Solution: I traced it to an inconsistency in how messages were stored. I standardized the data format across all three input types, and the crashes stopped.

Result

A reliable multimodal assistant that handles text, image analysis, and image generation — with fallback systems that keep it working even when external APIs fail.

Task 3: Agentic Medical Assistant Chatbot

What I Built

A medical question-answering chatbot that searches a local medical database first, falls back to live web search if needed, and uses a local Llama 3 model to generate a clear and accurate response — all while refusing to answer non-medical questions.

How It Works

- Medical terms are automatically extracted from your question using SciSpaCy, a medical NLP library.
- The system searches a local MedQA dataset using semantic similarity (HuggingFace's MiniLM model).
- If the local match isn't confident enough (below a 0.65 score), it automatically does a live web search using DuckDuckGo.
- A 'Gatekeeper' prompt screens every question and blocks anything non-medical (like recipe requests).
- The final answer is generated by Llama 3 running locally, which also checks the facts before responding.

Problems I Faced & How I Fixed Them

Problem: Google Gemini's free API kept hitting rate limits and crashing the bot mid-conversation.

Solution: I replaced the cloud API entirely with a locally hosted Llama 3 model. No rate limits, no data leaving the device.

Problem: Web search results were often garbage (cookie warnings, broken snippets), causing the AI to refuse to answer at all.

Solution: I changed the prompt so that instead of giving up on bad web results, the AI falls back to its own expert medical knowledge to still generate a helpful answer.

Problem: When the API timed out, the gatekeeper logic would fail in a way that let non-medical questions slip through and cause messy output.

Solution: I implemented a three-state gatekeeper: Medical, Non-Medical, or Timeout. On timeout, the app shows a clean 'System Alert' message instead of displaying junk.

Problem: The Streamlit app crashed on launch because it tried to display the chat before the session was initialized.

Solution: I reorganized the code so the session state is always set up before anything tries to use it.

Result

A privacy-first medical assistant that can explain complex topics like PCOS mechanisms or migraine phases, while refusing off-topic questions and never going down even under pressure.

Task 4: ArXiv Computer Science Expert Chatbot

What I Built

A fully offline chatbot that lets you ask questions about Computer Science research papers from Cornell University's ArXiv dataset — without any cloud API or internet connection needed during use.

How It Works

- The system processes a filtered subset of the ArXiv dataset, keeping only Computer Science papers.
- Abstracts are chunked and embedded using HuggingFace's MiniLM model, then stored in a local ChromaDB vector database.
- When you ask a question, the top 3 most relevant papers are retrieved and passed to a locally hosted Llama 3 model.
- The Streamlit interface shows the answer with expandable source dropdowns so you can see exactly which papers were used.

Problems I Faced & How I Fixed Them

Problem: The raw dataset had 1.4 million rows (over 1 GB) and crashed the machine when loaded all at once.

Solution: I used Pandas chunking to process the data in batches of 100,000 rows at a time, which kept memory usage manageable.

Problem: Non-CS papers (like physics papers) were sneaking into the dataset because their tags contained the letters 'cs'.

Solution: I used a strict Regular Expression pattern to match only exact Computer Science domain tags.

Problem: The Streamlit UI got stuck in a loop, showing an offline warning and duplicating messages.

Solution: I found that the issue was with how Streamlit handles empty chat input. I separated the system-readiness check from the chat input logic, which fixed the loop.

Problem: Version conflicts between Python packages caused import errors and VS Code showed ghost errors.

Solution: I did a clean reinstall of the LangChain ecosystem and ran the app directly through the local conda Python path to avoid conflicts.

Result

A zero-cost, private research assistant that can accurately summarize and cite CS papers on topics like graph decompositions or neural architectures — and safely refuses to answer questions outside its domain.

Task 5: Sentiment-Aware Customer Support Chatbot

What I Built

A customer support chatbot that reads the emotion behind a user's message and adjusts its tone accordingly — becoming more apologetic for frustrated users and more upbeat for happy ones.

How It Works

- User messages are analyzed using TextBlob, which gives each message a polarity score (positive, negative, or neutral).
- Based on the score, a dynamic prompt tells the local Llama 3 model exactly how to respond — empathetically for negative sentiment, warmly for positive.
- The sidebar shows the live polarity score and detected sentiment class so the user can see what the AI is detecting.

Key Challenges

Problem: Standard LLMs don't naturally adjust their tone — they'd respond to angry users in a generic, sometimes dismissive way.

Solution: I added a sentiment analysis step before the LLM receives the message. The result is injected into the system prompt as a behavioral instruction, forcing the model to match the user's emotional context.

Problem: The chat history and sentiment metrics kept resetting after each message.

Solution: I used Streamlit's session state to persistently store both the conversation and the latest sentiment data across interactions.

Result

A support chatbot that feels genuinely human — detecting emotional tone and responding in kind, which directly improves user satisfaction.

Task 6: Multilingual Sentiment-Aware Chatbot

What I Built

An extension of Task 5 that automatically detects which language the user is writing in and responds in that same language — with culturally appropriate phrasing for English, Gujarati, Hindi, and Spanish.

How It Works

- The langdetect library identifies the ISO language code from the user's input.
- A cultural mapping layer applies the right honorifics and phrasing for that language (e.g., using 'જી' or 'તમારું' for Gujarati).
- This language-aware instruction is combined with the sentiment guardrails from Task 5 into a single unified prompt.
- The prompt is sent to a locally hosted multilingual LLM for final response generation.

Key Challenges

Problem: Llama 3 completely failed at Gujarati — it produced broken output, mixed languages, and lost logical coherence.

Solution: I switched the underlying model from Llama 3 to Google's Gemma 2 (2B), which has much stronger training in Indian languages. This immediately fixed all the Gujarati and Hindi issues.

Problem: The app would occasionally show a blank screen if the model timed out, making it impossible to see what went wrong.

Solution: I identified a race condition with Streamlit's rerun function and moved the page refresh inside the try block, so errors stay visible on screen instead of silently disappearing.

Result

A culturally aware, emotionally intelligent chatbot that serves global users without requiring them to manually switch languages. A practical example of how the right model choice can make or break multilingual AI.

Skills & Competencies Gained

Area	Skills
AI / ML	Prompt Engineering, RAG Architectures, Multimodal AI, Named Entity Recognition
Vector Databases	ChromaDB, FAISS, Semantic Search, Embedding Management
NLP Libraries	LangChain, spaCy, SciSpaCy, TextBlob, langdetect
Software Engineering	Python (Advanced), Pandas, JSON, Cryptographic Hashing, Error Handling
Frontend Development	Streamlit — Session State, Layout Design, Reactive Components
Hardware Optimization	Local LLM Deployment (Ollama), VRAM Management, Model Quantization

Key Challenges & Solutions Summary

Challenge	Impact	Solution
Hardware Bottlenecks	Running Llama 3 (8B) and embeddings at the same time caused crashes.	Set Ollama to load only one model at a time; used Llama 3.2 (3B) on lower-spec machines.
File Sync Issues	Streamlit reloads caused the RAG system to forget already-indexed files.	Built a persistent JSON-based file tracker using MD5 hashes to track what's been processed.
API Rate Limiting	Google's image API kept throwing 429 errors.	Switched to Pollinations AI (Flux) and added a local image fallback to prevent UI freezing.
Context Confusion	The RAG system mixed up information from different user profiles.	Added automatic file boundary markers to every text chunk before passing it to the LLM.
'Always-On' Context	Asking 'How are you?' made the bot search CVs for an answer.	Set a distance threshold — if no close match is found, the bot answers from general knowledge.

Outcomes & Impact

The main output of this internship is a suite of six modular AI tools, each independently functional and ready to be integrated into larger enterprise systems.

By building everything to run locally, these projects demonstrate that data-sensitive industries — like healthcare, academic research, and HR — can use advanced AI without sending any private information to the cloud.

The sentiment-aware and multilingual bots also show how AI can be made more human — responding to how people feel, not just what they say, and working across languages without making users adapt to the tool.

Conclusion

This internship gave me real, hands-on experience at the intersection of software engineering and AI. Building six different chatbots pushed me to solve real engineering problems — not just follow tutorials.

I came away with deep practical knowledge of RAG systems, multimodal AI, emotional intelligence in conversational systems, and local LLM deployment. I'm grateful for the mentorship and resources that made this possible, and I'll carry these skills forward as I grow as an AI/ML engineer.

Appendix: Project Repositories

Task 1 — Dynamic RAG Chatbot: https://github.com/JineshKhalas/Dynamic_RAG_Chatbot

Task 2 — Multimodal Gemini Chatbot: https://github.com/JineshKhalas/Multimodel_Chatbot

Task 3 — Medical Assistant: https://github.com/JineshKhalas/MedicalQA_Chatbot

Task 4 — ArXiv Expert: https://github.com/JineshKhalas/Arxiv_Chatbot

Task 5 — Sentiment Support: https://github.com/JineshKhalas/Sen_Chatbot

Task 6 — Multilingual AI: https://github.com/JineshKhalas/Sen_Lang_Chatbot